# FlowCast: final technical report

Northline Corridor traffic forecasting. An internship project built from a client-style brief (PRD v1.0).

## 1. Summary

FlowCast forecasts volume, speed, travel time, congestion level and accident risk for 25 road segments, 30 to 120 minutes ahead, with a confidence estimate on each forecast. A Streamlit dashboard shows the results, and one command rebuilds everything from the raw files.

**Six of the eight targets below were met. Congestion and accident risk were missed.** All figures below are for the 30-minute horizon on the held-out test window (9 to 31 May 2025), with 95% intervals from resampling whole days.

| PRD target | Goal | Achieved | 95% interval | Result |
|---|---|---|---|---|
| Volume MAPE | at most 12% | 8.55% | 8.40 to 8.71 | Met |
| Sequence model beats best classical on RMSE | Below 54.62 | 54.27 | See section 6 | Met, narrowly |
| Congestion macro-F1 | at least 0.80 | 0.774 | 0.750 to 0.798 | **Not met** |
| Accident ROC-AUC | at least 0.75 | 0.643 | 0.608 to 0.683 | **Not met** |
| 80% confidence band coverage (added target) | 78 to 82% | 79.5% | | Met |
| Batch inference for one horizon | at most 30 s | About 3 to 5 s | | Met |
| One-command rebuild from raw data | Works | See section 10 | | Met |
| Dashboard shows all nine views on real outputs | Works | See section 10 | | Met |

Three findings matter most for anyone using this system:

1. **The misses come mainly from the data, not from a lack of tuning.** Congestion errors fall on band edges where forecast noise decides the class. Accident risk barely depends on anything a forecast can know in advance. Every model family landed within a narrow range on both targets.
2. **The deep model won, but didn't clearly earn its complexity.** It gained 1.2% on the classical model chosen on validation, at about 3 to 4 times the inference cost.
3. **Forecasts run too high in bad weather.** The bias is +37 vehicles in rain and +54 in fog, because a forecast can't know the weather in its target window.

The dataset looks synthetic: congestion is an exact function of volume, and travel time is an exact function of speed. So these results say little about real-world accuracy by themselves. **For volume only, the approach was also tested on real I-94 traffic, where it was off by 4.2% on average one hour ahead** (section 9).

## 2. Problem and data

The corridor team reacts to congestion after it forms. FlowCast is meant to give them 30 to 120 minutes of warning, with quantified confidence.

The data has 3 tables covering 25 segments from 1 January to 31 May 2025, in 30-minute windows.

| Fact | Value | Consequence |
|---|---|---|
| Traffic rows delivered | 178,468 | Rebuilt to the full grid of 181,200 |
| `vehicle_count` | Never null, equals volume on all valid rows | Bad volumes can be rebuilt exactly |
| `travel_time x speed / 60` | Constant per segment | Bad speeds can be rebuilt almost exactly |
| Congestion rule | Matches 100% of labelled rows | Congestion can be read directly off a volume forecast |
| Accident windows | 0.91% of measured windows | A rare target, and hard to rank |
| Fog and rain | Rare: most windows are dry and clear | Few examples of bad weather to learn from |
| Holidays and events | 6 of each | Effects are large but thinly evidenced |

## 3. Cleaning

The order follows PRD section 9.1. Every rejected row and changed value is one entry in `data/interim/quarantine.parquet`, with its rule and old and new values, and `reports/data_quality.md` is generated from that file.

| Defect | Entries | Resolution |
|---|---|---|
| Duplicate log rows | 1,767 | Deduplicated, keeping the most complete row |
| Absent windows | 4,499 | Added to the grid and estimated. Never used as targets or in metrics |
| Null or negative volume | 4,587 | Rebuilt from `vehicle_count` |
| Null or above-200 speed | 4,579 | Rebuilt from travel time and segment length |
| Occupancy above 100% | 229 | Blanked, then interpolated within the segment |
| Blank congestion labels | 31,123 | Every label recomputed; none disagreed with the rule |
| Inconsistent weather labels | 3,355 | Mapped to 4 values; an unknown label raises an error |

Statistical outliers are flagged and never changed, because traffic has real peaks and incidents. These figures match an independent profile of the raw files made before the pipeline was written.

Where a defect has an exact twin in another column, repairing from that column is better than interpolating. It's exact, and it means those rows can still count as measured.

## 4. Mathematics: where each concept lives

| Concept (PRD 10) | Where it's used |
|---|---|
| Matrices and dot products | `scratch.predict_linear` computes Xw. Neural layers compute Wx + b |
| Gradient descent | `scratch.gradient_descent_lr`, with the gradient (2/n)X^T(Xw - y) checked against finite differences |

| Concept (PRD 10) | Where it's used |
|---|---|
| Loss functions | MSE for regression. Cross-entropy for the multi-task head and classifiers |
| Feature scaling | Z-score for unbounded fields, min-max for bounded ones, all fit on training rows only |
| Covariance and correlation | `scratch.covariance`, used for the VIF check in the EDA |
| Probability | Accident probability with isotonic calibration, and congestion class probabilities |
| Normal distribution | The z-score outlier rule, and the residual check in the EDA (excess kurtosis 5.5, so errors aren't normal) |
| Evaluation metrics | Confusion matrix, precision, recall, F1 and ROC-AUC, each written in NumPy |

`tests/test_scratch.py` checks every one of these against scikit-learn or NumPy, to 1e-9 for metrics.

The from-scratch linear regression scored a test RMSE of 59.22, against 57.79 for the closed-form library version. The gap has a clear cause. Features are heavily collinear (VIF up to 282), so a fixed learning rate like 0.1 diverges. The step size has to come from the largest eigenvalue, and progress along the smallest-eigenvalue directions is slow within the 1,500-epoch budget. On well-conditioned test data, the same code matches the closed form to 1e-4.

## 5. Avoiding leakage

A forecast is issued at the end of window t, for window t+h. Only what's known by then may be used. Each rule below has a test in `tests/test_features.py` or `tests/test_dl_model.py`.

- **Lags and rolling statistics** are computed within each segment and end at t. A synthetic series where volume equals the window index proves this directly.
- **Changing the future doesn't change the past:** rewriting all later values leaves earlier features identical.
- **Target-window copies are excluded:** `vehicle_count`, `travel_time` and `congestion_level` at the target window never appear as inputs.
- **Weather** comes from the last completed hour at issue time. Using the target hour's weather would amount to a perfect weather forecast.
- **Scalers and imputation medians** use training dates only.
- **Splits are chronological:** train to 15 April, validation to 8 May, test to 31 May. Rows whose target crosses a boundary are dropped. Time-series CV uses expanding windows over whole days.
- **Estimated rows never count:** a row whose target was estimated is excluded from training targets and every metric.

## 6. Models and results

### Volume

| Model | Validation RMSE | Test RMSE | Test MAPE |
|---|---:|---:|---:|
| Persistence (last window) | 101.99 | 104.09 | 16.32% |
| Same window yesterday | 138.89 | 151.24 | 25.32% |
| Same window last week | 123.01 | 108.73 | 17.87% |
| Slot profile (usual level so far) | 75.59 | 78.21 | 12.58% |
| Linear regression, NumPy gradient descent | 57.41 | 59.22 | 10.95% |

| Model | Validation RMSE | Test RMSE | Test MAPE |
|---|---|---|---|
| Linear regression | 56.34 | 57.79 | 10.20% |
| Decision tree | 55.61 | 56.06 | 9.08% |
| Random forest (classical winner) | 55.00 | 54.93 | 8.66% |
| XGBoost | 55.72 | 54.62 | 8.72% |
| **LSTM (in service)** | **53.09** | **54.27** | **8.55%** |
| GRU | 53.27 | 54.28 | 8.66% |
| BiLSTM | 53.28 | 54.19 | 8.55% |
| LSTM with multi-task head | 53.58 | 54.13 | 8.58% |

**The feature that mattered most was added after the EDA.** Single-day lags are noisy, so I added a causal "usual level for this slot so far": an expanding mean of earlier values for the same segment, window and day type, skipping unusual days. On validation it cut XGBoost's RMSE from 59.6 to 55.4. Shuffling that group on test raises the forest's RMSE by 233, against 76 for recent volume and under 10 for every other group.

## How the models were chosen

Choices used validation and cross-validation only, under a written rule (PRD 11.4): take the best validation score, but prefer a faster model if it's statistically tied on both validation and CV, using the standard error of paired per-fold differences. `reports/model_selection.md` has the full tables. The winners were:

| Target | Model | Why it won |
|---|---|---|
| Volume | LSTM | Best validation RMSE of any model |
| Speed | Random forest | Same family as the classical volume winner (plan D7) |
| Travel time | Random forest | Lower validation RMSE than converting forecast speed (1.043 against 1.064) |
| Congestion | Tuned V/C bands on the volume forecast | Best validation macro-F1. The displayed class also stays consistent with the displayed volume, apart from slightly shifted cut-offs (0.49, 0.78, 0.99 instead of 0.50, 0.80, 1.00) |
| Accident risk | Logistic regression | Best validation ROC-AUC, and easy to read |

## Did the deep model earn its complexity?

**Not clearly.** The PRD's benchmark is met: the LSTM's test RMSE of 54.27 beats every classical model. Against the classical model chosen on validation, the gain is 0.67 vehicles per window, or 1.2%. The 95% interval for that gain, from resampling whole test days, is 0.28 to 1.13, so the gain is real. But I count a gain under 2% as too small to matter to operators, and the LSTM costs about 3 to 4 times as much per prediction, needs its full window of history, and can't be read through a simple feature ranking.

The search itself suggests a limit. Across 20 trials the validation RMSE stayed between 53.09 and 55.53, LSTM, GRU and BiLSTM are within 0.15 of each other on test, and test RMSE barely rises from 54.3 at 30 minutes to 55.3 at 120. Most of the remaining error looks like noise that no model can predict. For a simpler deployment, the random forest would be a reasonable choice, losing little accuracy.

The multi-task head didn't clearly help either. On validation, its congestion macro-F1 (0.765) matched reading the class off its own volume forecast (0.766), so it wasn't chosen. On test it was 0.776, within the noise of the 0.774 in service. BiLSTM reads its past window in both directions, which is still causal relative to

the target, and is shown as a context view on Historical trends.

### Congestion: why 0.80 wasn't reached

| | Forecast Free-flow | Moderate | Heavy | Severe |
|---|---|---|---|---|
| **Actual Free-flow** | 15,393 | 820 | 0 | 0 |
| **Actual Moderate** | 399 | 5,384 | 643 | 16 |
| **Actual Heavy** | 0 | 341 | 1,840 | 447 |
| **Actual Severe** | 0 | 0 | 582 | 1,004 |

Almost every error lands one class away. The F1 by class is 0.96 for Free-flow, 0.83 for Moderate, 0.65 for Heavy and 0.66 for Severe. The label is a fixed function of volume, so the only way to be wrong is for a forecast to fall on the wrong side of a boundary. The Heavy band is only 0.2 wide in V/C, while forecast error in V/C is around 0.08. Direct classifiers of every family reached 0.75 to 0.77, about the same as the band rule, which points to a limit in the data rather than the method. Even the upper end of the interval (0.798) falls short of the target.

### Accident risk: why 0.75 wasn't reached

Every family scored between 0.55 and 0.61 in cross-validation and validation (XGBoost's 0.614 CV was the highest), and tuning made little difference. Stronger models simply overfit: an early XGBoost reached 0.82 on training and 0.56 on validation. Even the rain actually observed in the target window gives a univariate AUC of only about 0.55.

The test score of 0.643 is higher than validation's 0.565, and I wouldn't read that as real improvement. The validation window had far less rain (2.8% of windows against 4.3%), and the accident signal is partly weather. The expected figure is probably around 0.60.

**The accident alert isn't usable as an alert.** The operating threshold was chosen for recall of at least 60%, as planned. On test it flags 69% of all windows at 1.15% precision, against a base rate of 0.98%. **The ranking does carry value:** the highest-scored 10% of windows see accidents 2.6 times as often as average. So the dashboard presents a ranked list, and says that the flag shouldn't be used alone.

## 7. Confidence estimates

Volume, speed and travel-time forecasts carry split conformal bands at 80% and 95%, calibrated on validation errors separately per horizon, peak or off-peak period and road-size group. The EDA justified the split: naive forecast error is more than twice as wide at peak.

| Test coverage, 30 minutes ahead | 80% band | 95% band |
|---|---|---|
| Volume | 79.5% | 94.6% |
| Volume at peak | 79.6% | 94.6% |
| Volume off-peak | 79.5% | 94.6% |
| Speed | 80.6% | 95.4% |
| Travel time | 79.8% | 94.6% |

Coverage stays within about 1 point of nominal at every horizon. Congestion probabilities come from how often validation errors would move a forecast into each band, and the reliability chart is on the Prediction confidence page.

## 8. Error analysis

These are test-window results for next-window volume.

- **By hour:** RMSE peaks at 8 to 9am (about 94) and is lowest overnight (about 18). MAPE stays near 8.5% across the day, so errors scale with volume.
- **By weekday:** weekends are easiest (39 to 42). Thursday is worst (65), and that window contains the Music Festival on 22 May.
- **By weather:** this is the most important finding for operators.

| Weather | Forecasts | RMSE | MAPE | Bias |
|---|---|---|---|---|
| Clear | 20,103 | 53.5 | 8.3% | -2.9 |
| Cloudy | 5,461 | 53.1 | 8.3% | +2.5 |
| Rain | 1,141 | 69.0 | 12.5% | +36.5 |
| Fog | 164 | 68.7 | 22.0% | +53.7 |

Rain and fog reduce demand, but the model can't know the weather in its target window, so it forecasts too much traffic. Shuffling the weather features barely changes volume accuracy (0.4 RMSE), because the last completed hour says little that recent volume doesn't already show. A live weather forecast input would be the most useful next addition.

## 9. Real-data validation

The Northline data looks synthetic, so the approach was also tested on real sensor data: hourly westbound traffic on I-94 in Minnesota, from the UCI Metro Interstate Traffic Volume dataset. The data was split before any model ran on it, with training to the end of 2016, validation on 2017, and testing on January to September 2018. Models were chosen on validation, and I reviewed those scores before the test window was scored.

**Result, one hour ahead:** the GRU neural network was off by 4.2% of traffic on average (WAPE, interval 4.0 to 4.4), with a test RMSE of 212 vehicles per hour and an R^2 of 0.988. Its error is 74% smaller than repeating the last hour, and the 80% range contained the actual value 80.5% of the time.

| Model | Test RMSE | Test WAPE |
|---|---|---|
| Same hour yesterday | 1,033.0 | 17.1% |
| Repeat the last hour | 811.8 | 17.7% |
| Same hour last week | 653.2 | 10.3% |
| Usual level so far | 526.8 | 9.4% |
| Linear regression | 321.7 | 6.8% |
| Random forest | 220.8 | 4.3% |
| XGBoost | 218.8 | 4.3% |
| **GRU (chosen on validation)** | **212.2** | **4.2%** |

Three things stand out:

- **The approach held up on messy data.** It had 7,629 duplicate hours, 11,976 absent hours including a gap of about ten months, and impossible readings such as temperatures of 0 Kelvin. The shared cleaning helpers, tuning, sequence models, confidence ranges and metrics all carried over. The features needed a new volume-only table with the same design, because this dataset has no speed, occupancy, capacity or accident columns.
- **The plan's prediction was wrong.** It expected larger errors on real data, and they were about half Northline's 8.3%. That doesn't mean real traffic is easier in general. This is one busy road counted hourly, averaging about 3,300 vehicles an hour, which smooths out random variation, so the two figures can't be

compared directly.
- **The weak spots repeat.** The neural network's lead over XGBoost is small again, and forecasts run too high in snow (+88 vehicles), just as they do in rain and fog on Northline. The weather weakness is therefore real, not a feature of the synthetic data.

**What it doesn't show:** this tests volume only. Congestion, travel time and accident risk remain untested on real roads. `reports/real_data.md` has the full results.

## 10. Reproducibility and verification

- **Rebuild:** I deleted all generated data and models, then ran `python -m src.pipeline all` from the raw files, which took about 80 minutes. Compared with the earlier run, the dataset hash and winning models are identical, every compared metric differs by 0.000000, and all 220,300 forecasts match (`reports/reproducibility.md`). That process exited with a non-zero code just after finishing, with no error logged. A retry with `--resume` reused the saved models and exited cleanly, and the cause was found later: on this machine, a process that has trained models can crash in native library cleanup (0xC0000409) once its work is done. The pipeline now ends without going through that cleanup.
- **Environment:** requirements are pinned. A new environment built from `requirements.txt` alone installs and passes the fast test suite.
- **Fresh clone:** the raw files and versioned dataset are in the repository. In a fresh clone from GitHub, the raw files were byte-identical to the originals, and rebuilding the dataset gave the same hash. The full model rebuild was run in the working copy, not in that clone.
- **Tests:** 131 fast tests need no data files, and 67 slow ones check the real data, saved outputs, model cards, this report's key figures, every dashboard page and the upload path. All pass.
- **Dashboard:** the tests run all 17 pages against real outputs, and I read each page in a browser. I ran the upload module with a test file containing deliberate defects (5.3 s), confirmed its forecasts reached another page and could be removed, and ran the retrain module, whose new model matched the one in service exactly (validation RMSE 54.9971). The report downloads were generated but not opened.

**Not verified:**

- The full pipeline ran on Windows only. On Linux, GitHub Actions runs lint and the fast tests, and rebuilds the dataset from the raw files, which gives the same hash as on Windows. Model training and the other slow tests haven't run there, and nothing has been run on macOS.
- The full rebuild ran with a CUDA build of PyTorch. CPU-only runs work, but will differ slightly in floating point.

## 11. What to know about how this was evaluated

- **A probe touched test data early.** While exploring accident signal, one script printed the test window's accident base rate and a rain-only AUC. No model was scored and nothing was tuned from it, but it was contact with the test window.
- **Test scores were computed more than once.** Each rerun reproduced the same scores unless noted here. A bug in choosing production models (a sequence model was fed rows with gaps, giving a false validation macro-F1 of 0.311) was found after the first test run, and fixing it changed the congestion model from the published bands to the tuned bands. That choice was made from validation numbers, and test macro-F1 moved from 0.771 to 0.774. Later reruns only added reporting. No model or threshold was changed in response to a test score.
- **The selection rule changed twice, before any test scoring.** Both changes came from seeing that it let a clearly worse model through on validation and CV results.
- **Real-data test scores were computed more than once too.** A first run scored the test window, and later runs repeated it to confirm the exit-code fix and the reproducibility. No choice changed.

- **Search budgets were reduced to fit the schedule:** 10 trials for XGBoost and tree models, 4 for random forest, and 3 for SVMs (trained on a 30,000-row subsample). The deep search kept its planned 20 trials. Larger searches are unlikely to change the conclusions, given how flat the results are.
- **Departures from the plan:** segment ID goes to tree models as a number rather than a categorical feature, and sequence inputs may read back across a split boundary so that the deep and classical models score identical rows. Most Beyond-PRD items (Optuna, MLflow and others) were cut for time. Real-data validation was done after the PRD deliverables were complete.

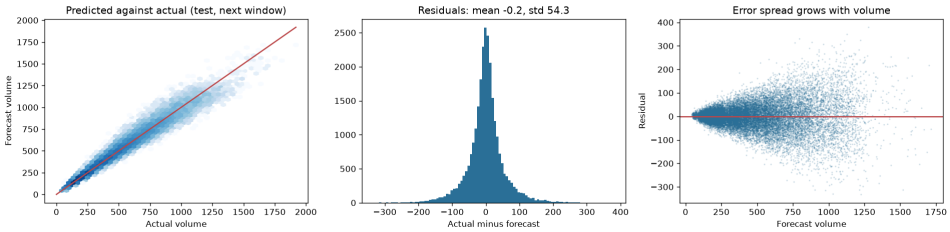
## 12. Limitations

- **The data looks synthetic,** so the Northline figures say little about real-world accuracy. The I-94 check covers volume on one road only.
- **Five months of data,** with 6 holidays and 6 events. Their effects are estimated from very few days.
- **One test window** of 23 days, in a single season.
- **Weather isn't forecast,** so bad-weather bias is built in.
- **Accident risk** is a weak ranking, and shouldn't be used to trigger responses by itself.

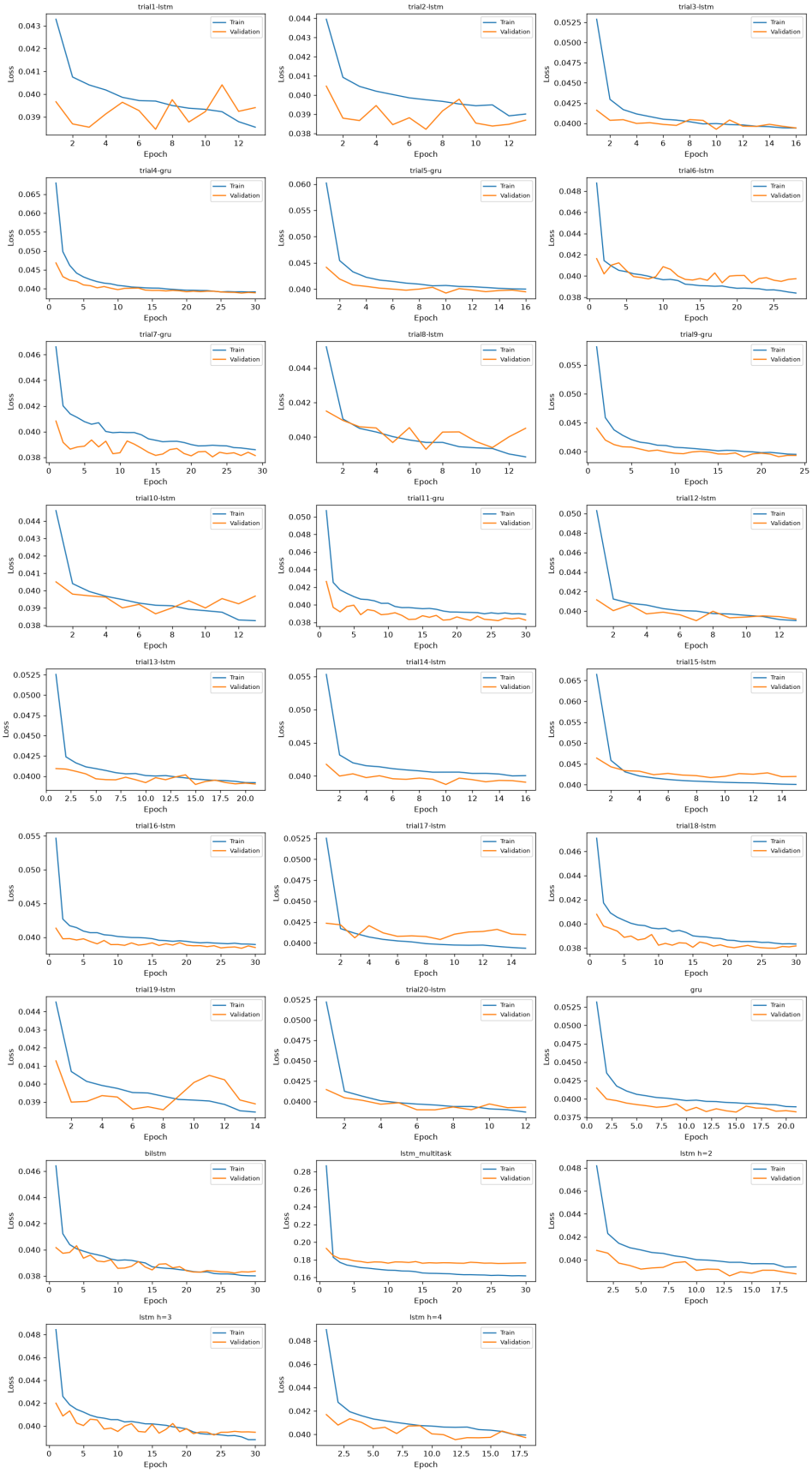
## 13. Recommendations and future work

1. **Add a weather forecast input.** It targets the largest known bias.
2. **Test congestion, travel time and accident risk on real data.** That needs a dataset with speed, capacity and incident records, which I-94 doesn't have.
3. **Use the accident model as a ranking only,** and have the operations team decide how many top-ranked windows they can act on.
4. **Consider serving the random forest** if operational simplicity matters more than a 1% gain.
5. **Roadmap from PRD 19:** streaming ingestion, graph-based models, automated retraining and drift monitoring. The segments' strong shared movement (median residual correlation 0.64) suggests graph models may be worth testing, though much of it may be shared day effects rather than traffic spreading between roads.

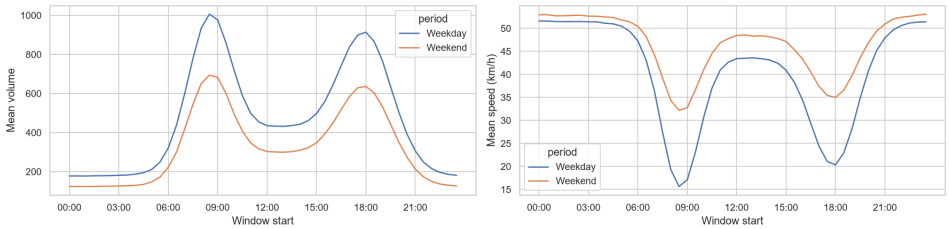
## Figures



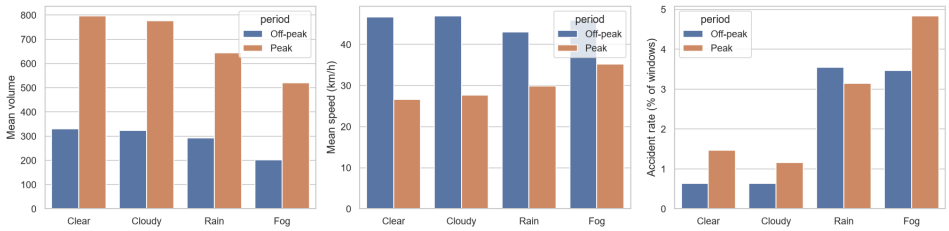

*Predicted against actual and residuals, test window, next-window volume.*




*Training and validation loss for every deep run.*




*Daily profile of volume and speed.*




*Volume, speed and accident rate by weather.*